Newmetric Postmortem & 문제 분석 기록

> 본 문서는 공개된 기술 레퍼런스와 직접 수행한 분석·실험을 토대로 작성되었으며, 회사의 기밀정보나 영업비밀을 포함하지 않습니다.

Newmetric에서 블록체인 인프라(Cosmos SDK 기반) 운영 중 발생한 장애 대응 및 문제 해결 사례 모음.

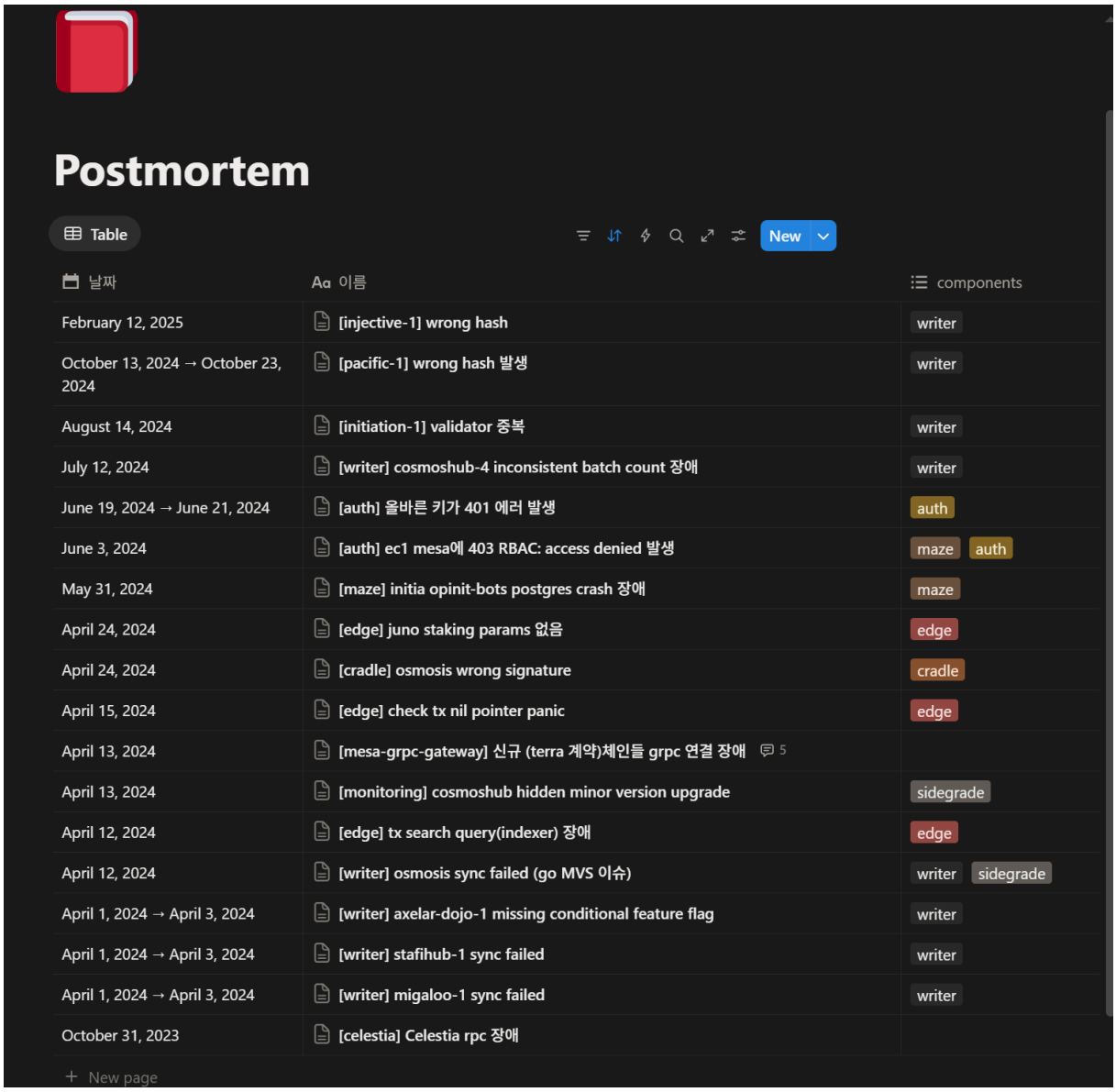
> 기간: 2023-10 ~ 2025-02

목차

1. 장애 대응 이력 Overview
2. Fiber Context 메모리 재사용으로 인한 캐시 키 변조
3. Pebble Batch Race Condition
4. gRPC `:authority` Header Bug
5. gRPC TLS/ALPN 협상 실패

---

1. 장애 대응 이력 Overview



Postmortem

Table

날짜	이름	components
February 12, 2025	[injective-1] wrong hash	writer
October 13, 2024 → October 23, 2024	[pacific-1] wrong hash 발생	writer
August 14, 2024	[initiation-1] validator 중복	writer
July 12, 2024	[writer] cosmoshub-4 inconsistent batch count 장애	writer
June 19, 2024 → June 21, 2024	[auth] 올바른 키가 401 에러 발생	auth
June 3, 2024	[auth] ec1 mesa에 403 RBAC: access denied 발생	maze auth
May 31, 2024	[maze] initia opinit-bots postgres crash 장애	maze
April 24, 2024	[edge] juno staking params 없음	edge
April 24, 2024	[cradle] osmosis wrong signature	cradle
April 15, 2024	[edge] check tx nil pointer panic	edge
April 13, 2024	[mesa-grpc-gateway] 신규 (terra 계약)체인들 grpc 연결 장애 5	
April 13, 2024	[monitoring] cosmoshub hidden minor version upgrade	sidegrade
April 12, 2024	[edge] tx search query(indexer) 장애	edge
April 12, 2024	[writer] osmosis sync failed (go MVS 이슈)	writer sidegrade
April 1, 2024 → April 3, 2024	[writer] axelar-dojo-1 missing conditional feature flag	writer
April 1, 2024 → April 3, 2024	[writer] stafihub-1 sync failed	writer
April 1, 2024 → April 3, 2024	[writer] migaloo-1 sync failed	writer
October 31, 2023	[celestia] Celestia rpc 장애	

New page

그림 1: postmortem-overview

2023-10 ~ 2025-02 동안 블록체인 인프라 운영 중 발생한 약 18건의 장애 이력.

주요 장애 유형

유형	사례
데이터 정합성	wrong hash, wrong signature, inconsistent batch count
환경/설정	missing feature flag, hidden minor version upgrade, Go MVS 이슈
런타임 에러	nil pointer panic, postgres crash
네트워크/프로토콜	gRPC 연결 장애, RPC 장애

---

2. Fiber Context 메모리 재사용으로 인한 캐시 키 변조

Timeline

06/19 오후 3시경: initia 팀으로부터 401에러가 발생한다는 리포트 발생

→ inita 팀의 키는 올바른 키 + policy 였음

재시작 이후 잘 통과되어서 안일하게 넘어감

06/21 오후 1시경: initia 팀으로부터 다시 리포트 받음

→ redis에 intia key에 다른팀의 policy + key 가 저장되어 있는 것을 확인

06/21 오후 3시 47분: `nm-authz/v2` 모듈에서 잘못된 key로 caching 되고 있음을 확인 및 수정

Problems

`fiber.Context` 는 zero allocation으로 handler cycle이 지나가면 메모리가 재활용됨

authz는 gocache 에서 loadable cache를 사용하는데, 이 로직은 setter buffered ch을 이용하여 loadfunc 로 부터 가져온 데이터를 cache에 백그라운드로 넣는 로직이 있고 아래와 같음

1. fiber ctx로 부터 추출하는 extractApiKey 에서 firstSubdomain 을 가져오는 로직이 ctx.Hostname() 을 타는데, 이건 fiber handler가 종료되면 재활용 되는 메모리

   -> https://github.com/newmetric/nm-authz/blob/08eea83bacc5a316afca3b3dfaa34332f9310f8b/server/authz/middlewares/key/middleware.go#L60-L75

2. go-cache 라이브러리에서 NewLoadable 이라는 constructor가 있는데, 기능 요약하면 cache에 없으면 loadFunc에서 가져오고, 가져온 데이터를 cache에 setter ch에 포인터로 전송 하는 로직

   -> https://github.com/eko/gocache/blob/78b7afe8bb7cd6e35ec38d4597b1ce096b71f847/lib/cache/loadable.go#L45-L59

3. 여기서 키가 1번에서 추출한 ctx.Hostname() 이라서 fiber.Context가 재활용되면 똑같은 메모리 크기의 다른 uuid키로 변경됨!

   setter ch가 buffered ch이고, 즉시 반영을 해주는게 아님 (in get func)

   cache에 store되기전에 request가 끝나면 psql에 가져온 값은 올바른 키여서 제대로된 value가 나오지만, redis에 저장되는 키는 변조 될 수 있음

Jiseok Choi 오후 3:47

authz 이상했던 문제 정리

1. fiber ctx로 부터 추출하는 extractApiKey 에서 `firstSubdomain` 을 가져오는 로직이 `ctx.Hostname()` 을 타는데, 이건 fiber handler가 종료되면 재활용 되는 메모리
   a. -> https://github.com/newmetric/nm-authz/blob/08eea83bacc5a316afca3b3dfaa34332f9310f8b/server/authz/middlewares/key/middleware.go#L60-L75
2. go-cache 라이브러리에서 `NewLoadable` 이라는 constructor가 있는데, 기능 요약하면 cache에 없으면 loadFunc에서 가져오고, 가져온 데이터를 cache에 setter ch에 포인터로 전송 하는 로직
   a. -> https://github.com/eko/gocache/blob/78b7afe8bb7cd6e35ec38d4597b1ce096b71f847/lib/cache/loadable.go#L45-L59
   b. 여기서 키가 1번에서 추출한 `ctx.Hostname()` 이라서 fiber.Context가 재활용되면 똑같은 메모리 크기의 다른 uuid키로 변경됨!!!
   c. setter ch가 buffered ch이고, setter가 즉시 반영을 해주는게 아님 (in get func)
   d. cache에 store되기전에 request가 끝나면 psql에 가져온 값은 올바른 키여서 제대로된 value가 나오지만, redis에 저장되는 키는 변조 될 수 있음

(편집됨)

2 2 2

4개의 댓글 2년 전 마지막 댓글

현상

- 인증/인가 모듈에서 특정 고객의 API 호출 시 **401 Unauthorized** 간헐 발생
- PostgreSQL에서 가져온 값은 정상이나, Redis 캐시에 저장되는 키가 **다른 팀의 UUID로 변조**되어 잘못된 정책(Policy)이 매핑됨

타임라인

시점	내용
06/19 15:00	고객으로부터 401 에러 리포트. 키/정책 정상 확인, 재시작 후 정상화되어 일시적 현상으로 판단
06/21 13:00	동일 리포트 재발. Redis에 해당 고객 키에 타 고객 정보가 저장된 것을 확인
06/21 15:47	캐시 키 변조 버그 확인 및 수정 완료

근본 원인

Fiber 프레임워크의 **zero-allocation 메모리 재사용**과 `eko/gocache`의 **비동기 setter 채널** 간 Race Condition.

1. **키 추출**: 미들웨어에서 `ctx.Hostname()`으로 subdomain(고객 UUID)을 캐시 키로 사용
2. **캐시 로드**: `NewLoadable`의 `loadFunc`으로 캐시 미스 시 PostgreSQL에서 데이터 획득 — 이 시점에는 `ctx.Hostname()` 값 유효
3. **비동기 저장**: 획득한 데이터를 `setter` **buffered channel**을 통해 백그라운드에서 Redis에 저장
4. **메모리 반환**: 응답 전송 후 핸들러 종료 → Fiber가 `ctx` 객체를 메모리 풀에 반환, 다음 요청에 재할당
5. **키 변조**: `setter` 채널이 실제 저장을 수행하는 시점에는 `ctx.Hostname()`이 가리키던 메모리가 **새로운 요청의 UUID로 덮어씌워짐** → 잘못된 키로 Redis에 캐시 저장

해결

`ctx.Hostname()` 반환값을 핸들러 내에서 **즉시 새로운 문자열로 복사(deep copy)** 하여 캐시 키로 사용. 핸들러 생명주기 이후에도 값 불변성 보장.

---

3. Pebble Batch Race Condition

Jiseok Choi 2024년 7월 12일 오후 3:17

https://github.com/newmetric/metricmint/issues/645#issuecomment-2224844987

> Comment on #645 [writer] IO fails when attempting to create a snapshot
>
> `pebble.Batch.Write(..)` & `pebble.DB.Open(..)` inconsistent batch count error
>
> batch에 동시에 Set/Delete operation을 call하는 상황
>
> > https://github.com/cosmos/cosmos-sdk/blob/d1b5b0c5ae2c51206cc1849e09e4d59986742cc3/store/pruning/manager.go#L170
> > 우리 디비는 lock이 없고, DB.Set/Delete를 Batch에 하는 atomic controller 가 있어서, SetSync를 호출하면 Batch.Set() 이 호출됨
>
> `pebble.Batch` 는 internal에 `batchInternal` 이라는 구조체가 있음
>
> > https://github.com/cockroachdb/pebble/blob/691154e9e3e642cb3967dccd05205a523e868c02/batch.go#L192-L313
>
> 이 구조체에서는 `data` 라는 `[]byte` type 의 필드가 있는데 이 값이 실제로 WAL에 쓰여지는 데이터임
>
> > https://github.com/cockroachdb/pebble/blob/691154e9e3e642cb3967dccd05205a523e868c02/batch.go#L1457-L1464
> > https://github.com/cockroachdb/pebble/blob/691154e9
> > ...
>
> 자세히 표시
>
> newmetric/metricmint | 2024년 7월 12일 | 봇이 추가한 GitHub

6개의 댓글

IH 2024년 7월 12일 오후 3:25

cc @orthory

Jiseok Choi 2024년 7월 12일 오후 3:25

tl;dr

> Batch에 동시 접근함

(편집됨)

ever0de on Jul 12, 2024 · edited by ever0de

Edits

`pebble.Batch.Write(..)` & `pebble.DB.Open(..)` inconsistent batch count error

batch에 동시에 Set/Delete operation을 call하는 상황

> cosmos/cosmos-sdk@ `d1b5b0c` /store/pruning/manager.go#L170
> 우리 디비는 lock이 없고, DB.Set/Delete를 Batch에 하는 atomic controller 가 있어서, SetSync를 호출하면 Batch.Set()이 호출됨

`pebble.Batch` 는 internal에 `batchInternal` 이라는 구조체가 있음

> cockroachdb/pebble@ `691154e` /batch.go#L192-L313

이 구조체에서는 `data` 라는 `[]byte` type 의 필드가 있는데 이 값이 실제로 WAL에 쓰여지는 데이터임

> cockroachdb/pebble@ `691154e` /batch.go#L1457-L1464
> cockroachdb/pebble@ `691154e` /mem_table.go#L204-L253

하지만 Batch Write에서 seqNum과 startSeqNum + (batch.Count() = batchInternal.count)를 비교하면서 잘못된 값이 들어오면 corruption을 내는데, 왜 DB.Open에서 다음과 같은 에러가 발생했을까?

> cockroachdb/pebble@ `691154e` /mem_table.go#L242-L246

우선 "DB.Open()" 에서 `pebble: inconsistent batch count: %d vs %d"` 에러가 날려면, 잘못된 WAL파일이 쓰여져야함 그렇다고하면 seqNum == startSeqNum + batch.Count() 는 동일하지만, `data의 header에 있는` data[:8]` 길이의 seqNum은 변경되었다는 말이 됨

> cockroachdb/pebble@ `691154e` /batch.go#L1399-L1404

타임라인상 아래와 같은 순서대로 진행되었을것임
---------Batch.Write()------ing...---check seq num---!!!insert new record!!!--- create wal file & fsync-----

그래서 Batch.Write는 넘어갔지만, 로컬에 잘못된 데이터가 적히고 디비가 손상되었을 것으로 예상됨
-> fatal 에러가 발생하고, pod이 재시작 되면서 DB.Open을 시도
WAL -> memtable.apply(batch)를 하는 와중에 잘못된 seqnum(이전에 batchInternal.data에 쓰여진 새로운 레코드 때문에)가 있어서 무한정 fail이 남

Action Item

- [ ] writer는 "싱글쓰레드에서 돌것"으로 예상하고, 동시에 write하는 루틴이 없다고 가정하는거라서 이 가정을 가지되 예외 케이스가 있을 수 있어서 mtx 를 추가

배경

Cosmos SDK v0.46 이전 버전까지 지원해야 하는 체인들은 tx execution 내부에서 별도의 goroutine을 태우지 않는 **싱글 스레드 순차 실행** 방식으로 구현되어 있었다. 그런데 SDK 버전이 올라가면서 `BaseApp.Commit` 시점에 **특정 interval마다 스냅샷을 생성하는 기능**(`go app.snapshotManager.SnapshotIfApplicable(header.Height)`)이 goroutine으로 비동기 실행되도록 변경되었고, 이로 인해 기존의 싱글 스레드 전제가 깨졌다.

현상

- `pebble.Batch.Write()` 및 `pebble.DB.Open()` 호출 시 **inconsistent batch count error** 발생
- 런타임 중 데이터 모순이 발견되어 **Fatal Error → Pod 종료**
- 재시작 시 WAL 복구 과정에서도 오염된 데이터로 인해 DB Open 실패 → **무한 재시작 루프(Infinite Fail Loop)**

근본 원인

`pebble.Batch`는 **thread-safe하지 않음**. 기존에는 싱글 스레드로만 접근되어 lock 없이 동작했으나, SDK 버전 업그레이드로 비동기 스냅샷 goroutine이 추가되면서 여러 goroutine이 동시에 `Batch.Set/Delete`를 호출하게 됨.

1. **데이터 오염**: `Batch.Write()` 진행 중 다른 goroutine이 `Set/Delete`를 호출하면 내부 `data []byte` 버퍼와 레코드 카운트(`count`)가 불일치
2. **WAL 오염**: `data[:8]`에 저장되는 시퀀스 넘버 헤더가 쓰기 도중 수정되어, 오염된 상태로 `fsync` 완료
3. **복구 불가**: Pod 재시작 시 `pebble.DB.Open()`에서 WAL을 읽어 Memtable에 복구 시도 → 오염된 헤더와 실제 레코드 수 불일치 → `pebble: inconsistent batch count` 에러로 DB Open 실패

해결

싱글 스레드 전제를 폐기하고, `Batch` 객체 접근 시 **Mutex**를 추가하여 상호 배제 보장.

---

4. gRPC `:authority` Header Bug

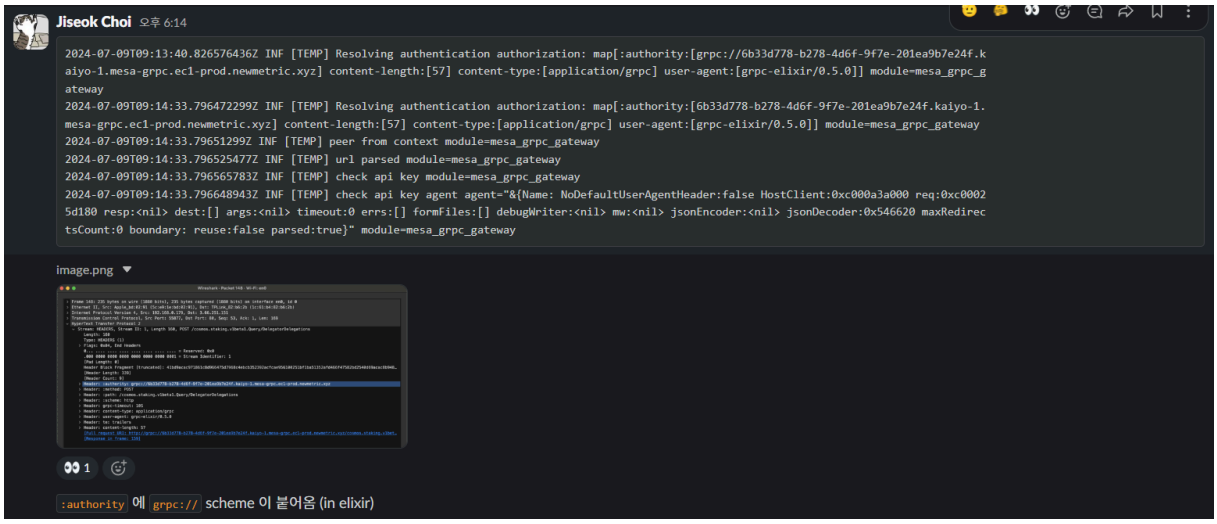


그림 2: grpc-authority-header-bug

현상

- Elixir 기반 gRPC 클라이언트(`grpc-elixir/0.5.0`)가 `:authority` 헤더에 `grpc://` 스킴을 접두어로 포함하여 전송
- 정상 기대값: `host:port` → 실제 전달값: `grpc://host:port`
- gRPC 게이트웨이에서 인증 처리 시 파싱 실패

분석 방법

1. **서버 로그 분석**: gRPC 게이트웨이에서 `:authority` 필드에 `grpc://` 문자열 포함 확인
2. **Wireshark 패킷 캡처**: HTTP/2 프레임에서 `:authority` 헤더에 스킴이 중복 포함된 규격 위반 교차 검증
3. **Elixir 재현 코드**: gRPC 인터셉터로 raw response 확인

재현

Elixir `grpc-elixir` 라이브러리가 `:authority`에 `grpc://` 스킴을 붙이는 것을 재현하기 위해 별도 프로젝트 작성.

- `GRPC.ClientInterceptor`를 구현한 커스텀 인터셉터로 raw gRPC response를 캡처하여 헤더 확인

- Cosmos SDK 체인의 `DelegatorDelegations` RPC를 호출하면서 `:authority` 헤더에 수동으로 값을 삽입해 동작 비교
- 결과: `grpc-elixir`의 `scheme: "grpc"` 옵션이 `:authority`에 스킴을 접두어로 붙이는 것을 확인

해결

gRPC 게이트웨이의 인증 미들웨어에서 `:authority` 헤더로부터 고객 UUID(첫 번째 subdomain)를 추출하는 로직에, `"://"` 기준으로 split하여 scheme이 포함되어 있으면 strip하는 처리를 추가.

```
// before: host가 "grpc://uuid.chain.example.com"이면
// Split(host, ".")[0] → "grpc://uuid" → UUID parse 실패
firstSubdomain := strings.Split(host, ".")[0]

// after: scheme 존재 여부를 먼저 확인하고 제거
splitHosts := strings.Split(host, "://")
if len(splitHosts) > 1 {
    host = splitHosts[1]
}
firstSubdomain := strings.Split(host, ".")[0]
```

---

5. gRPC TLS/ALPN 협상 실패

현상

- 고객이 Python gRPC 클라이언트로 gRPC 엔드포인트에 TLS 연결 시 통신 실패
- `openssl s_client -alpn h2`로 확인 결과 **"No ALPN negotiated"** — HTTP/2 ALPN 협상이 이루어지지 않음

분석 방법

- Python(`cosmpy` + `grpc`)으로 `ssl_channel_credentials`를 통한 secure channel 연결을 재현
- `openssl s_client`로 TLS 핸드셰이크 시 ALPN 프로토콜 협상 결과를 직접 확인

재현

```
openssl s_client -connect <endpoint>:443 -alpn h2 | grep ALPN
# No ALPN negotiated
```

해결

TLS 종단(로드밸런서/프록시)에서 HTTP/2 ALPN 프로토콜을 지원하도록 설정 수정.